

PART A — Analysis of the model & correct use of $\text{MAX} = 5$

Finite version of model (with $\text{MAX} = 5$ or better 1 to obtain better graph)

This is the **minimal, correct fix**, written in proper CSP-M style.

```
-- =====
-- Finite bounds for model checking
-- =====
MAX = 5
BAL = {0..MAX}

-- =====
-- Events / channels
-- =====
channel deposit_small, deposit_large, withdraw, query, clear_large,
fraud_withdraw
channel show_balance : BAL

-- =====
-- Bank account process
-- avail : available funds
-- held  : funds on hold
-- =====
ACCOUNT(avail, held) =

  -- Small deposit: immediate availability
  (avail < MAX & deposit_small ->
    ACCOUNT(avail + 1, held))

  []

  -- Large deposit: goes on hold
  (held < MAX & deposit_large ->
    ACCOUNT(avail, held + 1))

  []

  -- Normal withdrawal (only if funds available)
  (avail > 0 & withdraw ->
    ACCOUNT(avail - 1, held))

  []

  -- Balance query
  query ->
    show_balance!avail ->
    ACCOUNT(avail, held)

  []
```

```

-- Clear held funds (fraud check passed)
(held > 0 & avail + held <= MAX & clear_large ->
  ACCOUNT(avail + held, 0))

[]

-- Fraudulent withdrawal (bypasses controls)
(avail > 0 & fraud_withdraw ->
  ACCOUNT(avail - 1, held))

-- =====
-- Initial state
-- =====
BANK = ACCOUNT(0, 0)
MAIN = BANK

```

- $\text{avail} \in \{0..5\}$
- $\text{held} \in \{0..5\}$
- Total states are **finite**
- FDR can now:
 - build the LTS
 - show communication graphs
 - check assertions
 - avoid “Evaluating process” deadlock

PART B —FDR prompt commands (:help)

Core analysis commands

:processes

Lists all defined processes.

You want to confirm that `MAIN`, `BANK`, etc. were parsed correctly.

:graph <PROCESS>

Displays the **Labelled Transition System (LTS)** of a process.

Example:

```
:graph MAIN
```

This is the main “analysis” view in FDR.

:communication_graph <PROCESS>

Shows how processes communicate via events.

Example:

```
:communication_graph MAIN
```

```
:structure <PROCESS>
```

Shows the **syntactic structure** of a process (how it is composed).

Process composition, hiding, parallelism.

Assertion-related commands

```
:assertions
```

Lists all assertions found in the CSP file and their status.

Example assertion:

```
assert MAIN :[deadlock free]
```

```
:run <ASSERTION>
```

Runs a specific assertion manually.

Used when:

- You have many assertions
- You want to control execution

```
:counterexample <ASSERTION>
```

Shows the **counterexample trace** if an assertion fails.

Very important for:

- security modelling
- refinement failures

```
:debug <ASSERTION>
```

Interactive debugging of a failed assertion.

Advanced use, mostly for:

- refinement checking
- deep analysis

```
:statistics <ASSERTION>
```

Shows statistics such as:

- number of states
- transitions
- time taken

Exploration & inspection

```
:probe <PROCESS>
```

Allows **manual step-by-step exploration** of transitions.

Closest thing to “simulation” in FDR, but still symbolic.

```
:type <EXPRESSION>
```

Prints the **type** inferred by CSP-M.

Example:

```
:type MAIN
```

Useful for debugging type errors.

Environment / utility commands

```
:load <file>
```

Loads a CSP file.

:reload

Reloads the current file.

:cd

Changes directory.

:options

View or change tool options.

:profiling_data

Shows profiling info (if enabled).

:version

Prints FDR version.

:quit

Exits FDR.

What a Communication Graph actually is

In FDR, a **communication graph** shows:

- **multiple processes**
- **running in parallel**
- synchronising on shared events

Example structure:

```
CLIENT ||| SERVER
CLIENT [| {|withdraw|} |] SERVER
```

It answers questions like:

- *Which process sends which event?*
- *Where do processes synchronise?*
- *What is the communication structure?*

For communication graph you must introduce **parallel structure**.

Example: split the system into components

```
-- ATM interface
ATM =
  deposit_small -> ATM
  []
  withdraw -> ATM

-- Bank backend
BACKEND(avail) =
  deposit_small -> BACKEND(avail+1)
  []
  (avail > 0 & withdraw -> BACKEND(avail-1))

SYSTEM =
  ATM [] {|deposit_small, withdraw|} [] BACKEND(0)
```

Now you can run:

```
:communication_graph SYSTEM
```

Because:

- ATM and BACKEND are **separate processes**
- They **communicate via shared events**

Bank model WITH parallel composition

We decompose the system into **three parallel components**:

- **CLIENT** (ATM / user actions)
- **BANK_CORE** (account logic)
- **FRAUD_MONITOR** (can allow or bypass controls)

They synchronise on shared events.

```

-- =====
-- Finite bounds for model checking
-- =====
MAX = 5
BAL = {0..MAX}

-- =====
-- Channels
-- =====
channel deposit_small, deposit_large, withdraw, fraud_withdraw
channel query, clear_large
channel show_balance : BAL

-- =====
-- Client (ATM / user interface)
-- =====
CLIENT =
    deposit_small -> CLIENT
[]
    deposit_large -> CLIENT
[]
    withdraw -> CLIENT
[]
    fraud_withdraw -> CLIENT
[]
    query -> CLIENT

-- =====
-- Bank core logic
-- =====
BANK_CORE(avail, held) =

    (avail < MAX & deposit_small ->
        BANK_CORE(avail + 1, held))

[]

    (held < MAX & deposit_large ->
        BANK_CORE(avail, held + 1))

[]

    (avail > 0 & withdraw ->
        BANK_CORE(avail - 1, held))

[]

    query ->
        show_balance!avail ->
            BANK_CORE(avail, held)

[]

    (held > 0 & avail + held <= MAX & clear_large ->
        BANK_CORE(avail + held, 0))

-- =====
-- Fraud monitor

```

```

-- =====
FRAUD_MONITOR =
  clear_large -> FRAUD_MONITOR
  []
  fraud_withdraw -> FRAUD_MONITOR

-- =====
-- Parallel system
-- =====
SYSTEM =
  CLIENT
    [| {deposit_small, deposit_large, withdraw, fraud_withdraw,
query|} |]
  BANK_CORE(0, 0)
    [| {fraud_withdraw, clear_large|} |]
  FRAUD_MONITOR

MAIN = SYSTEM

```

Parallel composition structural view

Structural view (conceptual)

```
CLIENT || BANK_CORE || FRAUD_MONITOR
```

Synchronisation rules

Event	Who participates
deposit_small	CLIENT ↔ BANK_CORE
deposit_large	CLIENT ↔ BANK_CORE
withdraw	CLIENT ↔ BANK_CORE
query	CLIENT ↔ BANK_CORE
fraud_withdraw	CLIENT ↔ BANK_CORE ↔ FRAUD_MONITOR
clear_large	BANK_CORE ↔ FRAUD_MONITOR

This is **exactly** what a communication graph is meant to show.

Why this now qualifies as a “high-level machine”

A **high-level machine** = a process built from **parallel compositions**

You now have:

- multiple components

- explicit synchronisation sets
- real inter-process communication

```
:communication_graph MAIN
```

What to run in FDR

After loading the file:

```
:processes
```

You should see:

```
CLIENT
BANK_CORE
FRAUD_MONITOR
SYSTEM
MAIN
```

Then:

```
:communication_graph MAIN
```

You should now get a **visual graph** showing:

- the three processes
- edges labelled with shared events

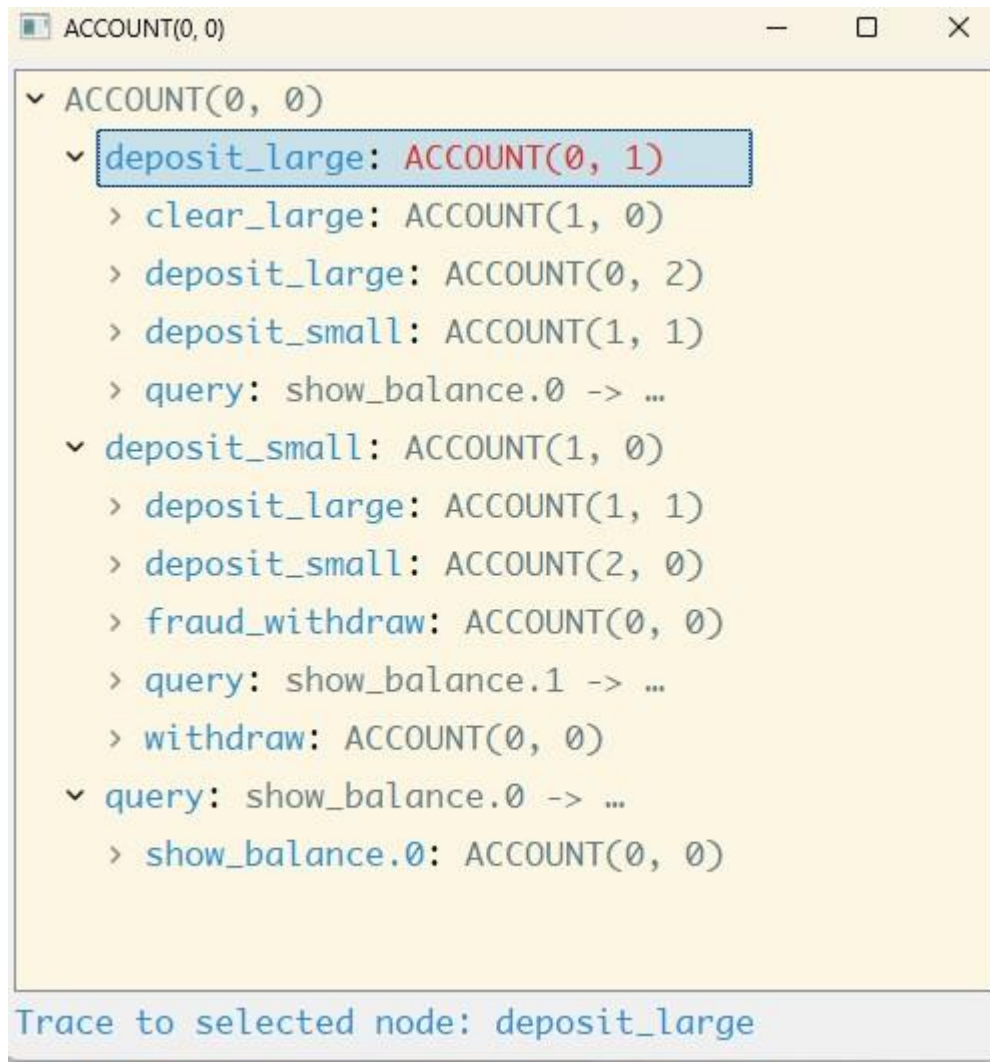
Also useful:

```
:graph MAIN
```

This version provides:

- explanation of **distributed systems**
- show **security as a cross-cutting concern**
- demonstrate **how fraud bypasses normal controls**
- contrast:
 - sequential vs parallel models
 - behaviour vs structure

Analysis of figures:



1. What :probe MAIN does

:probe puts FDR into **interactive exploration mode**.

Conceptually:

“Starting from the initial state, show me **all possible next events**, and if I choose one, show me the next state and its possible events, etc.”

So :probe is the **closest thing to a step-by-step simulation**, but:

- it is **symbolic**
- it shows **all possible behaviours**, not just one execution

2. What the tree you see represents

You are probing this process:

```
MAIN = ACCOUNT(0,0)
```

So the **root node** is:

```
ACCOUNT(0,0)
```

Meaning:

- avail = 0
- held = 0

Every indented branch corresponds to **one enabled event** from that state.

3. Interpreting the first-level branches

From the screenshot, at `ACCOUNT(0,0)` you see (among others):

```
deposit_large : ACCOUNT(0,1)
deposit_small : ACCOUNT(1,0)
query : show_balance.0 -> ACCOUNT(0,0)
```

3.1 `deposit_large : ACCOUNT(0,1)`

From the code:

```
deposit_large ->
  ACCOUNT(avail, held + 1)
```

At `(0,0)`:

- avail = 0
- held = 0

After `deposit_large`:

- avail = 0
- held = 1

Exactly matches `ACCOUNT(0,1)`

3.2 `deposit_small : ACCOUNT(1,0)`

From the code:

```
deposit_small ->
```

```
ACCOUNT(avail + 1, held)
```

At (0,0):

- avail becomes 1
- held stays 0

Exactly matches ACCOUNT(1,0)

3.3 query : show_balance.0 -> ACCOUNT(0,0)

From the code:

```
query ->
  show_balance!avail ->
  ACCOUNT(avail, held)
```

At (0,0):

- avail = 0

So the interaction is:

1. query
2. output show_balance.0
3. return to the same state

That is why you see:

```
query : show_balance.0 -> ACCOUNT(0,0)
```

This is a **two-step interaction**, which the probe shows compactly.

4. Why withdraw and fraud_withdraw do NOT appear at (0,0)

Look at the guards in code:

```
(avail > 0 & withdraw -> ...)
(avail > 0 & fraud_withdraw -> ...)
```

At (0,0):

- avail = 0
- condition `avail > 0` is **false**

So:

- withdraw is **disabled**
- fraud_withdraw is **disabled**

Correctly not shown in the probe

This confirms that **guards are working exactly as intended**.

5. Going one level deeper

```
deposit_large : ACCOUNT(0,1)
```

From (0,1) you now see options like:

```
clear_large : ACCOUNT(1,0)
deposit_large : ACCOUNT(0,2)
deposit_small : ACCOUNT(1,1)
query : show_balance.0 -> ...
```

Let's interpret the key one:

```
clear_large : ACCOUNT(1,0)
```

From the code:

```
(held > 0 & clear_large ->
  ACCOUNT(avail + held, 0))
```

At (0,1):

- avail = 0
- held = 1

After clear_large:

- avail = 0 + 1 = 1
- held = 0

Exactly ACCOUNT(1,0)

This confirms:

- the **“hold then clear” logic** is modelled correctly
- large deposits are not immediately withdrawable

6. What the probe is *telling you conceptually*

The probe view is showing that the model satisfies these intended properties:

1. **Funds on hold are not withdrawable**
2. **Withdrawals are blocked at zero balance**
3. **Balance queries reveal only available funds**
4. **Fraud withdrawals bypass normal checks (when enabled)**
5. **State transitions exactly match the financial logic**

In other words:

The operational semantics of the CSP model **represent your intended bank-account behaviour.**

7. Relation to MAX=5 and BAL

MAX = 5
BAL = {0..MAX}

What you are seeing confirms that:

- states remain **bounded**
- FDR can fully explore the model
- no infinite evaluation occurs

If MAX were missing, :probe would eventually block or explode.

1 One normal trace (step by step)

We start from the initial state:

ACCOUNT (0, 0)

Meaning:

- avail = 0
- held = 0

Step 1: deposit_large

From your probe:

deposit_large : ACCOUNT (0, 1)

Interpretation:

- A large deposit is made
- Money is **on hold**, not available

State now:

```
ACCOUNT (0, 1)
```

Step 2: `clear_large`

From the probe expansion:

```
clear_large : ACCOUNT (1, 0)
```

Interpretation:

- Fraud check completes
- Held funds are released

State now:

```
ACCOUNT (1, 0)
```

Step 3: `withdraw`

From the model:

```
(avail > 0 & withdraw -> ACCOUNT(avail - 1, held))
```

Enabled because `avail = 1`.

State now:

```
ACCOUNT (0, 0)
```

Step 4: `query`

From probe:

```
query : show_balance.0 -> ACCOUNT (0, 0)
```

Interpretation:

- User queries balance
- System outputs 0
- State does not change

Trace summary (what FDR would show)

```
<deposit_large, clear_large, withdraw, query, show_balance.0>
```

2A fraud trace (security-relevant)

Now let's demonstrate **malicious behaviour**.

Start again at:

```
ACCOUNT(0,0)
```

Step 1: `deposit_small`

```
deposit_small : ACCOUNT(1,0)
```

Step 2: `fraud_withdraw`

From your model:

```
(avail > 0 & fraud_withdraw -> ACCOUNT(avail - 1, held))
```

This bypasses normal checks.

State becomes:

```
ACCOUNT(0,0)
```

Fraud trace:

```
<deposit_small, fraud_withdraw>
```

This trace shows:

- money was withdrawn
- **outside the normal withdrawal channel**
- without going through legitimate banking logic

This is what you want when modelling **security violations**.

3 Assertions & counterexamples

3.1 Deadlock freedom

Add to your file:

```
assert MAIN :[deadlock free]
```

Expected result: PASSES

Why?

- There is always at least one enabled event (`query`, `deposits`, etc.)
- No state blocks completely

3.2 Safety property: “No fraud allowed”

Define a safe version:

```
ACCOUNT_SAFE(avail, held) =  
  (avail < MAX & deposit_small ->  
    ACCOUNT_SAFE(avail + 1, held))  
  []  
  (held < MAX & deposit_large ->  
    ACCOUNT_SAFE(avail, held + 1))  
  []  
  (avail > 0 & withdraw ->  
    ACCOUNT_SAFE(avail - 1, held))  
  []  
  query ->  
    show_balance!avail ->  
    ACCOUNT_SAFE(avail, held)  
  []  
  (held > 0 & avail + held <= MAX & clear_large ->  
    ACCOUNT_SAFE(avail + held, 0))  
  
SAFE = ACCOUNT_SAFE(0,0)
```

Now check refinement:

```
assert MAIN [T= SAFE
```

Expected result: FAILS

FDR will produce a **counterexample trace**, e.g.:

```
<deposit_small, fraud_withdraw>
```

This formally proves:

MAIN allows behaviours that SAFE does not
⇒ MAIN is insecure relative to SAFE

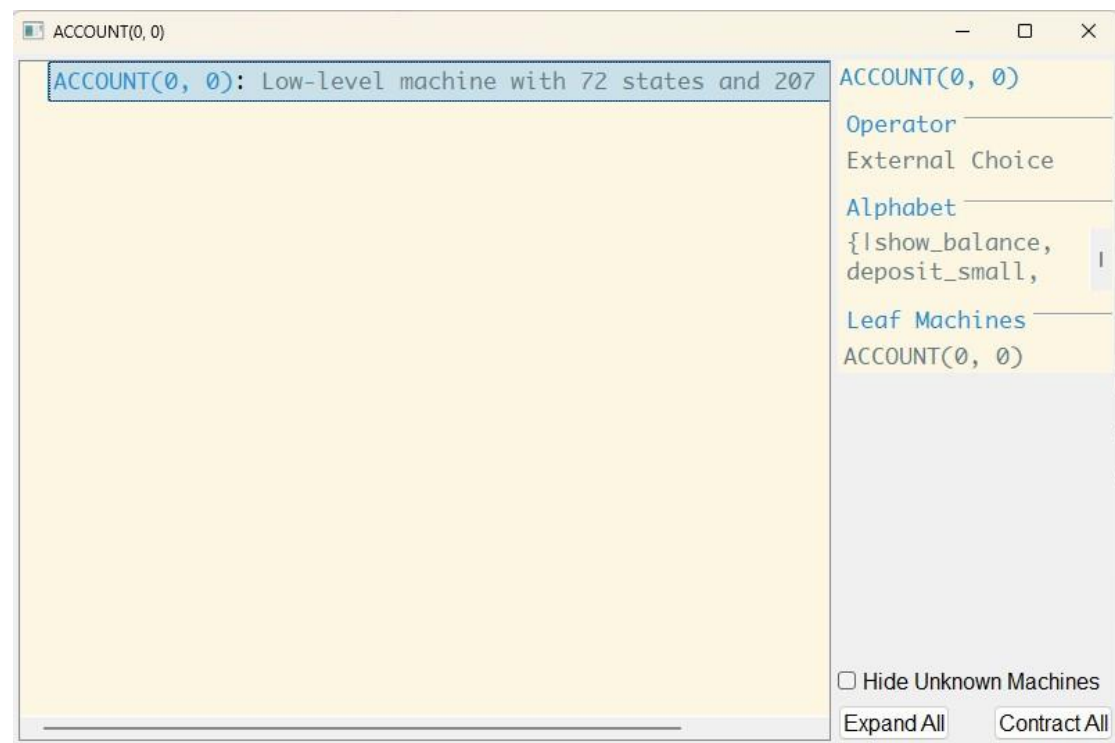
4 How to explain this to students

Each node is a system state (`avail`, `held`).
Each outgoing edge is an event that is *currently enabled*.
If an event is missing, its guard is false.”

- Guards = access control
- States = account balances
- Traces = possible executions
- Counterexamples = attacks

That’s:

- why `withdraw` is absent at `(0,0)`
- why `clear_large` only appears when `held > 0`
- why `query` does not change state
- how `fraud_withdraw` bypasses policy



: structure

Τι είναι το :structure στο FDR

Το :structure ΔΕΝ δείχνει συμπεριφορά (traces)

Δείχνει τη συντακτική / δομική μορφή του process όπως την καταλαβαίνει η FDR.

Από τι είδους CSP τελεστές αποτελείται αυτό το process;

Τι παρουσιάζεται στο figure

Στο πάνω μέρος γράφει:

`ACCOUNT(0, 0): Low-level machine with 72 states and 207 transitions`

Αυτό σημαίνει:

Low-level machine

- Το `ACCOUNT(0, 0)` έχει **μεταγλωττιστεί** σε ένα *state machine*
- Δεν είναι πια “υψηλού επιπέδου CSP”, αλλά **καθαρό LTS**
- Αυτό είναι προαπαιτούμενο για model checking

72 states

- Υπάρχουν **72 διαφορετικές καταστάσεις** (`avail`, `held`)
- Προκύπτουν από:
 - τα όρια `MAX = 5`
 - τους `guards`
 - τις δυνατές αλληλουχίες γεγονότων

207 transitions

- Υπάρχουν **207 δυνατές μεταβάσεις**
- Κάθε transition = ένα event (`deposit`, `withdraw`, `query`, κ.λπ.)

Αυτό δείχνει ότι:

- το state space είναι **πεπερασμένο**
- η FDR μπορεί να το εξερευνήσει πλήρως
- το μοντέλο σου είναι **σωστά περιορισμένο**

Τι σημαίνουν τα panels δεξιά

1 Operator: External Choice

Η FDR λέει ότι το `ACCOUNT` είναι δομημένο ως:

P [] Q [] R [] ...

Δηλαδή:

- **External choice** ([1])
- Το περιβάλλον (user / ATM / attacker) επιλέγει ποιο event θα συμβεί

Αυτό ταιριάζει με τραπεζικό μοντέλο:

- ο πελάτης επιλέγει αν θα κάνει deposit, withdraw, query κ.λπ.

2 Alphabet

Στο panel εμφανίζεται:

```
{show_balance, deposit_small, deposit_large, withdraw, query,
clear_large, fraud_withdraw}
```

Αυτό είναι το **αλφάβητο του process**.

Σημαίνει:

- αυτά είναι **όλα** τα events που *μπορεί ποτέ* να κάνει το ACCOUNT
- αν ένα event **δεν** είναι εδώ → δεν μπορεί να συμβεί

3 Leaf Machines

```
Leaf_Machines:
  ACCOUNT(0, 0)
```

Αυτό σημαίνει:

- δεν υπάρχουν εσωτερικά υπο-processes
- το ACCOUNT είναι **ατομικό state machine**
- δεν έχει parallel composition μέσα του

Γι' αυτό **ΔΕΝ** μπορεί να έχει communication graph από μόνο του.

Πώς συνδέεται αυτό με όσα εμφανίζονται στο :probe

Το :probe έδειξε:

- τοπικά **enabled events**
- state-by-state συμπεριφορά

Το `:structure` δείχνει:

- ολόκληρο το **state space**
- πόσο μεγάλο είναι
- πώς δομείται συντακτικά

Μαζί δηλώνουν ότι:

- ✓ Το μοντέλο είναι σωστό
- ✓ Είναι finite
- ✓ Είναι fully explorable
- ✓ Οι guards δουλεύουν

- “*To structure δείχνει τι είναι το σύστημα*”
- “*To LTS δείχνει πώς συμπεριφέρεται*”
- “*Ta traces δείχνουν τι μπορεί να συμβεί*”
- “*Ta counterexamples δείχνουν τι πάει στραβά*”

Το μοντέλο υποστηρίζει **και τα τέσσερα**.

Σύνδεση με ασφάλεια (πολύ σημαντικό)

Από αυτό το structure μπορείς να αιτιολογήσεις:

- γιατί βάλαμε MAX
- γιατί χωρίς MAX το state space είναι άπειρο
- γιατί το `fraud_withdraw` αυξάνει transitions
- γιατί η ασφάλεια = περιορισμός συμπεριφοράς

1. What this paragraph is really about (big picture)

The passage explains how **basic CSP prefixing**:

$a \rightarrow P$

is **extended** to support **communication of data values**, not just simple events.

This extension is what allows CSP to model:

- message passing
- parameters

- state-dependent interaction
- higher-order information flow

In other words:

CSP events are not just names — they can *carry values*.

2. Prefix choice → communication channels

Basic prefixing

In plain CSP:

$a \rightarrow P$

means:

“The process performs event a , then behaves like P .”

This is **control flow** — no data.

Extending prefixing with channels

The text introduces **channels**:

A **channel** is a *family of events*, indexed by values.

3. Formal definition explained

The key definition:

$c.T = \{ c.x \mid x : T \} \subset \Sigma$

Let’s unpack this carefully.

Components

- $c \rightarrow$ channel name
- $T \rightarrow$ type (set of values)
- $\Sigma \rightarrow$ the **alphabet of all events** in the system

Meaning

The channel c with type T represents **all events of the form $c.x$** , where x ranges over values of type T .

So if:

$$T = \{0, 1, 2\}$$

then:

$$c.T = \{ c.0, c.1, c.2 \}$$

Each $c.x$ is a **distinct atomic CSP event**.

4. Input and output intuition (very important)

Output (sending a value)

In CSP-M:

$$c!v \rightarrow P$$

means:

Output value v on channel c , then continue as P .

Formally, this is the event $c.v$.

Input (receiving a value)

$$c?x \rightarrow P(x)$$

means:

Accept *any* value $x \in T$ on channel c , then behave like $P(x)$.

This expands to an **external choice** over all $c.x$.

5. Why this is called “higher-order information”

Without channels:

- events are just labels
- processes can only synchronise on *names*

With channels:

- events carry **data**
- behaviour can depend on **values**
- processes can exchange **state information**

6. Example (bank model)

From your CSP model:

```
channel show_balance : BAL
```

This defines:

```
show_balance.BAL = { show_balance.0, show_balance.1, ...,  
show_balance.5 }
```

Then in the process:

```
query ->  
  show_balance!avail ->  
  ACCOUNT(avail, held)
```

This means that

- query synchronises with the environment
- show_balance!avail **outputs the current balance**
- the actual event is show_balance.0, show_balance.1, etc.

7. Why this matters for modelling and verification

This extension allows CSP/FDR to model:

- client-server protocols
- authentication challenges
- bank balances
- security tokens
- sensor values
- API responses

All while preserving:

- trace semantics
- failures semantics
- refinement checking

Because **each value is still a concrete event**.

Conceptual Analysis of input in CSP

1. Conceptual distinction: input vs output in CSP

The key idea in the passage is that **input** and **output** over a channel play **asymmetric roles** in CSP.

- **Input** expresses *willingness* to accept any value
- **Output** expresses *commitment* to a specific value

This asymmetry is fundamental to CSP's semantics.

2. Inputting process: $c?x:T \rightarrow P(x)$

2.1 Syntactic meaning

$c?x:T \rightarrow P(x)$

The process is willing to engage in an event $c.x$ for **any** value x drawn from the set T , and after that it behaves as $P(x)$.

2.2 Semantic expansion

Formally, this construct the basic for an *external choice*:

$[] \ x:T \ @ \ c.x \rightarrow P(x)$

Which means:

- The environment chooses which value $x \in T$ is communicated
- The process does **not restrict** the value
- The process must be prepared for *all* possibilities